

# Proyecto de Arquitectura en la Nube: Administración Remota de Servidores Web en AWS

**Título: Despliegue y Configuración de Múltiples  
Servidores Web (Apache, Nginx, Caddy) en una  
Instancia EC2**

**Autor: Enrique Crespo Ramírez**

**Rol: Administrador de Sistemas en la Nube**



# 1. Introducción y Conceptos Fundamentales de AWS

El objetivo de este proyecto es simular un entorno de administración de sistemas real. Hemos desplegado y configurado múltiples servidores web (Apache, Nginx y Caddy) en una instancia **EC2** de AWS, y hemos gestionado todo el proceso de forma remota a través de **SSH** desde una terminal local (WSL).

## 1.1. Componentes Clave de la Arquitectura

- **AWS (Amazon Web Services):** Es el proveedor de servicios en la nube que utilizamos para alquilar nuestro "ordenador virtual".
- **EC2 (Elastic Compute Cloud):** Es el servicio que nos permite crear y ejecutar esa máquina virtual, eligiendo el sistema operativo (Ubuntu, Debian, etc.) y sus recursos.
- **SSH (Secure Shell):** El protocolo cifrado que utilizamos para acceder a la terminal de la Instancia EC2 de forma segura, como si estuviéramos sentados frente a ella.
- **Clave PEM:** El archivo que contiene nuestra clave privada criptográfica. Es la **llave digital** necesaria para autenticarnos ante la Instancia EC2 y establecer la conexión SSH. Debe estar siempre protegida.
- **Security Group:** Actúa como un *firewall* virtual a nivel de AWS. Controla qué tráfico de red se permite *entrar* o *salir* de la Instancia EC2. Es crucial para la seguridad.

## 2. Fase de Preparación Local (WSL y Clave PEM)

Antes de conectarnos a AWS, debemos preparar nuestro entorno local (WSL) para gestionar la clave de acceso de forma segura.

### 2.1. Verificación de WSL y Creación del Directorio SSH

- **Comando ejecutado:** `wsl --version`
- **Explicación:** Se verifica que el Subsistema de Windows para Linux (WSL) esté funcionando correctamente, ya que será la herramienta que usaremos para ejecutar el cliente SSH.

- **Resultado:** Se confirma la versión de WSL y el kernel de Linux.

```
C:\Windows\System32>wsl --version
Versión de WSL: 2.6.1.0
Versión de kernel: 6.6.87.2-1
Versión de WSLg: 1.0.66
Versión de MSRDC: 1.2.6353
Versión de Direct3D: 1.611.1-81528511
Versión de DXCore: 10.0.26100.1-240331-1435.ge-release
Versión de Windows: 10.0.26100.5074
```

*Verificación de la versión de WSL en Windows para asegurar la compatibilidad con el cliente SSH.*

## 2.2. Gestión y Permisos de la Clave PEM

- **Comandos clave:**
  - `mkdir -p ~/.ssh`
  - `chmod 700 ~/.ssh`
  - `cp /mnt/c/.../labsuser.pem ~/.ssh/`
  - `chmod 400 ~/.ssh/labsuser.pem`
- **Explicación:** Se creó el directorio oculto `.ssh` en el entorno Linux local con permisos restrictivos (700). Luego, se movió la clave PEM descargada de AWS (ej. `labsuser.pem`) a ese directorio. Finalmente, el comando `chmod 400` es crucial: establece permisos de **solo lectura para el propietario** y **ningún permiso para otros usuarios**. SSH rechazará la clave si estos permisos no son estrictos.

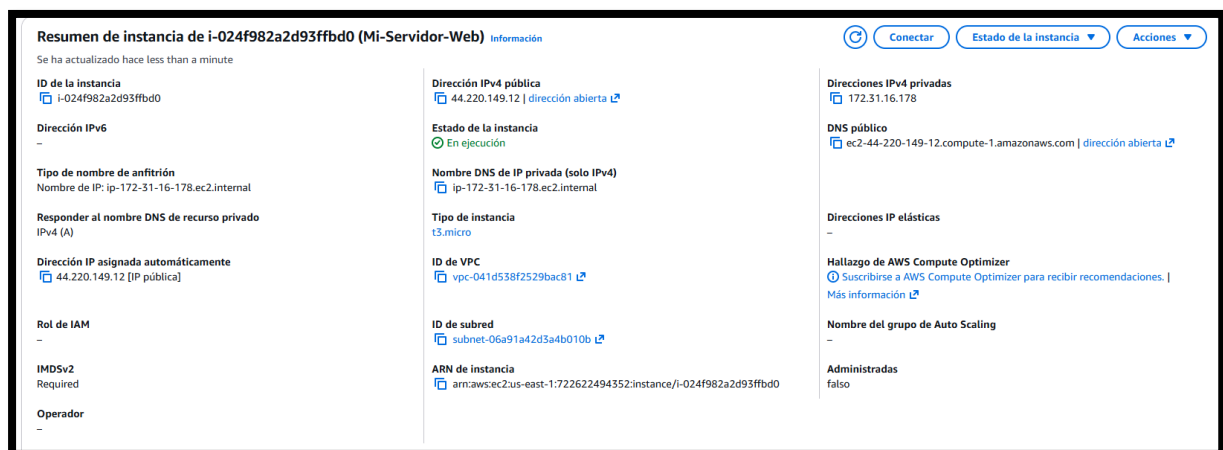
```
kike@A6Alumno09:/mnt/c/Windows/System32$ ls -l ~/.ssh/
total 4
-r----- 1 kike kike 1674 Nov 12 10:51 labsuser.pem
```

*Listado de directorios (`ls -l ~/.ssh/`) mostrando la clave PEM con permisos estrictos de solo lectura para el propietario (`-r-----`).*

## 3. Fase de Configuración en AWS (Instancia y Red)

### 3.1. Creación y Descripción de la Instancia EC2

- **Configuración:** Se lanzó una Instancia EC2 utilizando la AMI (Amazon Machine Image) de **Ubuntu 24.04 LTS** y un tipo de instancia **t2/t3.micro** (dentro de la capa gratuita).
- **Explicación:** La instancia se nombra y se le asigna una clave pública (extraída de nuestra clave PEM) que se utiliza para la autenticación remota.



**Resumen de la Instancia EC2 creada, mostrando su ID, la dirección IPv4 pública y el estado Running.**

### 3.2. Configuración del Grupo de Seguridad (Security Group)

- **Propósito:** Actuar como el *firewall* de entrada a la instancia EC2.
- **Reglas configuradas:** Se crearon 5 reglas de entrada (Inbound Rules) para permitir la conexión a todos los servicios web que vamos a desplegar, además de SSH.
  - **22 (SSH):** Para la administración remota.
  - **8080 (HTTP):** Para el primer servidor Apache.
  - **8081 (HTTP):** Para el servidor Nginx.
  - **8082 (HTTP):** Para el servidor Caddy.
  - **8443 (HTTPS):** Para el tráfico cifrado de Apache.
- **Explicación:** Si no abrimos estos puertos en el Security Group de AWS, aunque los servidores estén instalados en la EC2, el tráfico se bloquearía antes de llegar a la instancia.

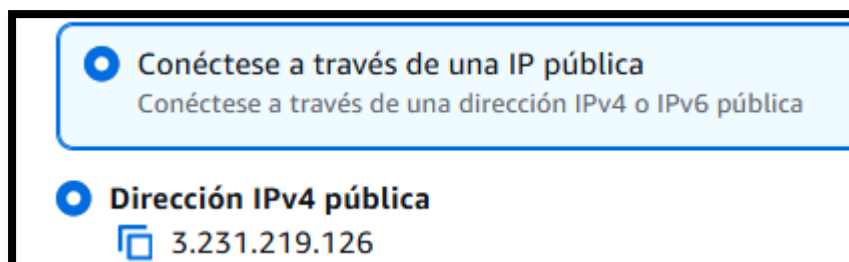
| Buscar                   | Name               | ID de la regla del gr... | Versión de IP | Tipo              | Protocolo | Intervalo de puertos | Origen    | De |
|--------------------------|--------------------|--------------------------|---------------|-------------------|-----------|----------------------|-----------|----|
| <input type="checkbox"/> | Nginx              | sgr-0ba2effea127560a7    | IPv4          | TCP personalizado | TCP       | 8081                 | 0.0.0.0/0 | -  |
| <input type="checkbox"/> | Apache HTTPS (SSL) | sgr-0f45439b1c45037ea    | IPv4          | TCP personalizado | TCP       | 8443                 | 0.0.0.0/0 | -  |
| <input type="checkbox"/> | Caddy              | sgr-0cc15f83ca2509706    | IPv4          | TCP personalizado | TCP       | 8082                 | 0.0.0.0/0 | -  |
| <input type="checkbox"/> | Apache HTTP        | sgr-060bbe29a1d063f76    | IPv4          | TCP personalizado | TCP       | 8080                 | 0.0.0.0/0 | -  |
| <input type="checkbox"/> | Acceso SSH Remoto  | sgr-08bf6ad65b2fd2ec9    | IPv4          | SSH               | TCP       | 22                   | 0.0.0.0/0 | -  |

**Tabla de reglas de entrada del Security Group, confirmando los 5 puertos abiertos (22, 8080, 8081, 8082 y 8443) a 0.0.0.0/0.**

## 4. Fase de Conexión SSH y Acceso Remoto

### 4.1. Obtención de IP Pública

- **Acción:** Se obtuvo la dirección IPv4 pública de la Instancia EC2 desde la consola de AWS.



**Dirección IPv4 pública de la Instancia EC2, necesaria para establecer la conexión remota.**

### 4.2. Conexión SSH Exitosa

- **Comando ejecutado:** `ssh -i ~/.ssh/labsuser.pem ubuntu@TU_IP_PÚBLICA`
- **Explicación:** Se inicia la conexión SSH. La opción `-i` especifica la ruta a la clave privada (`labsuser.pem`), y se utiliza el usuario por defecto de la AMI Ubuntu (`ubuntu`) junto con la IP pública del servidor.
- **Resultado:** El mensaje de bienvenida de Ubuntu confirma que hemos accedido exitosamente a la terminal remota de la Instancia EC2.

```
kike@A6Alumno09:~/ssh$ ssh -i ~/.ssh/mi-servidor-web2.pem ubuntu@3.231.219.12
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Nov 13 09:11:36 UTC 2025

System load:  0.0           Temperature:   -273.1 C
Usage of /:   26.2% of 6.71GB Processes:    114
Memory usage: 22%          Users logged in: 1
Swap usage:   0%           IPv4 address for ens5: 172.31.76.39

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Thu Nov 13 08:42:11 2025 from 18.206.107.29
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

**Conexión SSH exitosa a la Instancia EC2 en AWS, abriendo la terminal remota de Ubuntu.**

## 5. Fase de Despliegue y Configuración de Servidores Web

Una vez conectados a la terminal remota, se replicaron los pasos de la segunda práctica para instalar y configurar tres servidores web distintos, asignando a cada uno un puerto de escucha diferente.

### 5.1. Servidor Apache (Puerto 8080)

#### 5.1.1. Instalación y Verificación

- **Comandos:** `sudo apt update`, `sudo apt install apache2 -y`, `sudo systemctl status apache2`.

- **Explicación:** Se instala el servidor Apache2 y se verifica que el servicio se encuentre **active (running)**.

```

ubuntu@ip-172-31-76-39:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 08:18:49 UTC; 1h 7min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 520 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Process: 939 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
  Main PID: 559 (apache2)
    Tasks: 55 (limit: 1017)
   Memory: 10.2M (peak: 12.2M)
      CPU: 331ms
   CGroup: /system.slice/apache2.service
           └─559 /usr/sbin/apache2 -k start
             └─978 /usr/sbin/apache2 -k start
               └─979 /usr/sbin/apache2 -k start

Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started apache2.service - The Apache HTTP Server.
Nov 20 08:18:50 ip-172-31-76-39 systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Nov 20 08:18:50 ip-172-31-76-39 systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-76-39:~$

```

**Estado del servicio Apache2 confirmado como activo y en ejecución.**

### 5.1.2. Cambio de Puerto

- **Ficheros editados:** /etc/apache2/ports.conf y /etc/apache2/sites-available/000-default.conf.
- **Explicación:** Por defecto, Apache escucha en el puerto 80. Se modificaron los ficheros de configuración (ports.conf y 000-default.conf) para que Apache escuche en el puerto **8080** y se reinició el servicio (sudo systemctl restart apache2). La imagen que subiste confirma esta edición.

```

GNU nano 7.2
<VirtualHost *:8080>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>

```

**Configuración del VirtualHost de Apache en `000-default.conf` y el archivo `ports.conf` para escuchar en el puerto 8080.**

```

# /etc/apache2/sites-enabled/000-default.conf

Listen 8080

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>

```

### 5.1.3. Comprobación del Puerto de Escucha

- **Comando ejecutado:** `sudo ss -tlnp | grep 8080`
- **Explicación:** El comando `ss` verifica qué puertos están siendo escuchados por los servicios activos. Se confirma que el proceso de Apache (`apache2`) está escuchando correctamente en el puerto 8080.

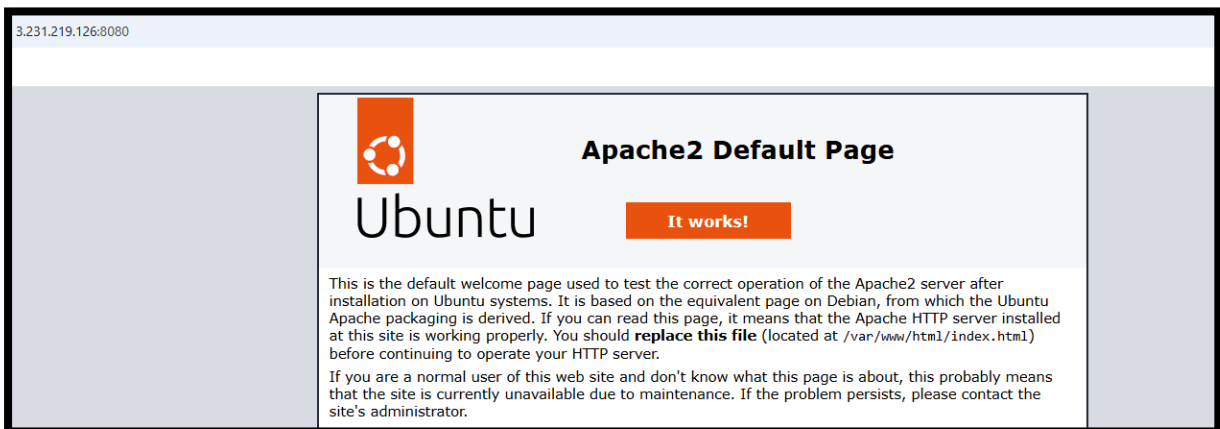


```
# /etc/apache2/sites-enabled/000-  
Listen 8080  
  
<IfModule ssl_module>  
    Listen 443  
</IfModule>  
  
<IfModule mod_gnutls.c>  
    Listen 443  
</IfModule>
```

**Comando SS confirmando que Apache está escuchando en el puerto 8080.**

#### 5.1.4. Acceso desde el Navegador

- **URL de Acceso:** [http://TU\\_IP\\_PÚBLICA:8080](http://TU_IP_PÚBLICA:8080)
- **Explicación:** Se accede desde el navegador local utilizando la IP pública de la Instancia EC2 y el puerto 8080, confirmando que la configuración del servidor y el Security Group son correctos.



**Página por defecto de Apache ("It works!") accesible a través del puerto 8080.**

## 5.2. Servidor Nginx (Puerto 8081)

### 5.2.1. Instalación y Configuración

- **Comandos:** `sudo apt install nginx -y, sudo nano /etc/nginx/sites-available/default.`
- **Explicación:** Se instaló Nginx, y se modificó el archivo de configuración por defecto para cambiar el puerto de escucha de 80 a **8081**.

```
# Please see /usr/share/doc/nginx-doc/examples/ for more det
##

# Default server configuration
#
server {
    listen 8081 default_server;
    listen [::]:8081 default_server;

    # SSL configuration
    #
    # listen 443 ssl default_server;
    # listen [::]:443 ssl default_server;
    #
    # Note: You should disable gzip for SSL traffic.
    # See: https://bugs.debian.org/773332
```

### 5.2.2. Comprobación del Puerto de Escucha

- **Comando ejecutado:** `sudo ss -tlnp | grep 8081`
- **Explicación:** Se verifica que Nginx haya reiniciado correctamente y esté escuchando en el puerto 8081.

```
Docs: man:nginx(8)
Process: 3945 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Process: 3947 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Main PID: 3982 (nginx)
Tasks: 3 (limit: 1008)
Memory: 2.4M (peak: 5.3M)
CPU: 26ms
CGroup: /system.slice/nginx.service
├─3982 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
├─3984 "nginx: worker process"
└─3985 "nginx: worker process"

Nov 13 09:53:48 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 13 09:53:48 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-76-39:~$ sudo nano /etc/nginx/sites-available/default
ubuntu@ip-172-31-76-39:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-76-39:~$ sudo ss -tlnp | grep 8081
LISTEN 0      511          0.0.0.0:8081      0.0.0.0:*        users:((("nginx",pid=4079,fd=5),("nginx",pid=4078,fd=5),("nginx",pid=4077,fd=5))
LISTEN 0      511          [::]:8081        [::]:*           users:((("nginx",pid=4079,fd=6),("nginx",pid=4078,fd=6),("nginx",pid=4077,fd=6))
ubuntu@ip-172-31-76-39:~$
```

**Comando ss verificando que Nginx está escuchando en el puerto 8081.**

```
o VM guests are running outdated hypervisor (qemu) binaries on this host.
buntu@ip-172-31-76-39:~$ systemctl status nginx
nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
Active: active (running) since Thu 2025-11-13 09:53:48 UTC; 15s ago
Docs: man:nginx(8)
Process: 3945 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Process: 3947 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
Main PID: 3982 (nginx)
Tasks: 3 (limit: 1008)
Memory: 2.4M (peak: 5.3M)
CPU: 26ms
CGroup: /system.slice/nginx.service
├─3982 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
├─3984 "nginx: worker process"
└─3985 "nginx: worker process"

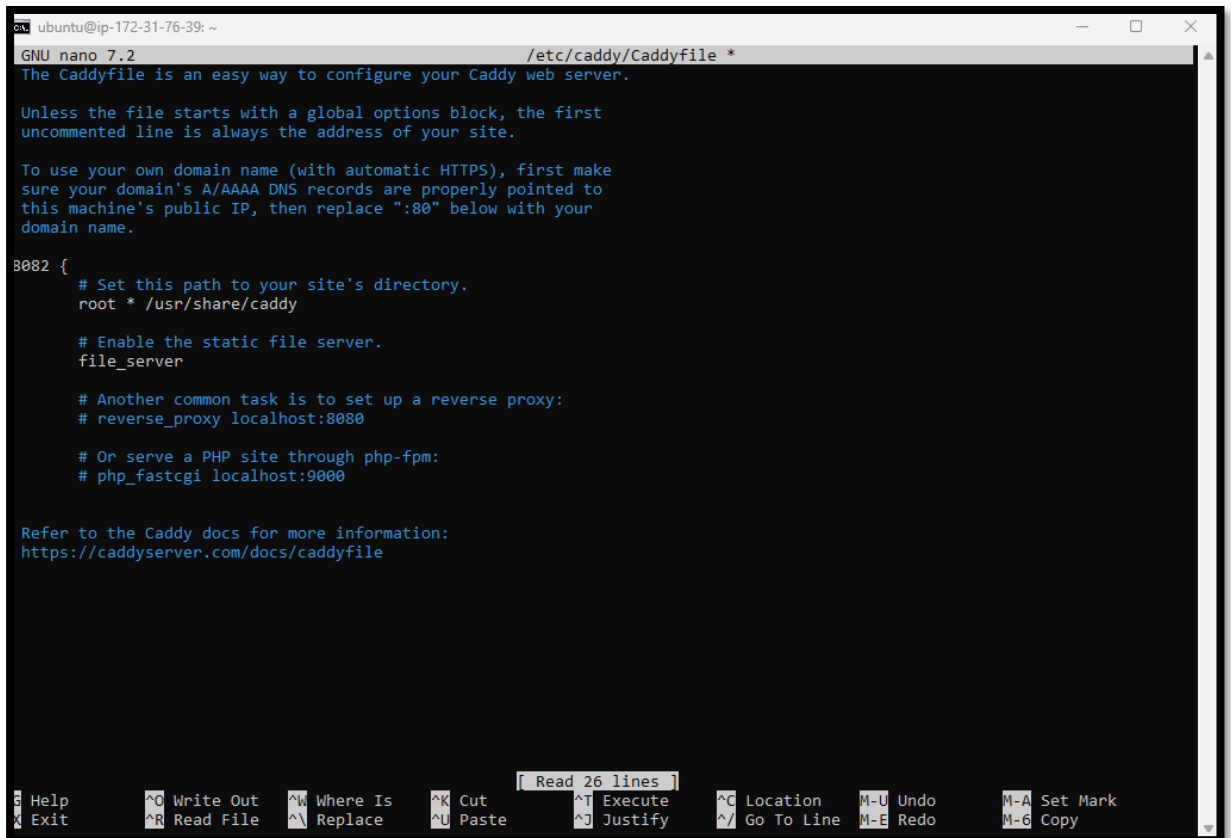
ov 13 09:53:48 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
ov 13 09:53:48 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

**Nginx funcionando correctamente**

## 5.3. Servidor Caddy (Puerto 8082)

### 5.3.1. Instalación y Configuración

- **Comandos:** Se realizaron los pasos para agregar el repositorio oficial de Caddy e instalarlo (`sudo apt install -y caddy`). Luego se editó el Caddyfile (`/etc/caddy/Caddyfile`) para cambiar el puerto de escucha a **8082**.



The screenshot shows a terminal window with the nano text editor open, editing the file `/etc/caddy/Caddyfile`. The terminal title bar indicates the user is `ubuntu@ip-172-31-76-39: ~`. The nano editor header shows `GNU nano 7.2` and the file path `/etc/caddy/Caddyfile *`. The content of the file is as follows:

```
The Caddyfile is an easy way to configure your Caddy web server.

Unless the file starts with a global options block, the first
uncommented line is always the address of your site.

To use your own domain name (with automatic HTTPS), first make
sure your domain's A/AAAA DNS records are properly pointed to
this machine's public IP, then replace ":80" below with your
domain name.

8082 {
    # Set this path to your site's directory.
    root * /usr/share/caddy

    # Enable the static file server.
    file_server

    # Another common task is to set up a reverse proxy:
    # reverse_proxy localhost:8080

    # Or serve a PHP site through php-fpm:
    # php_fastcgi localhost:9000

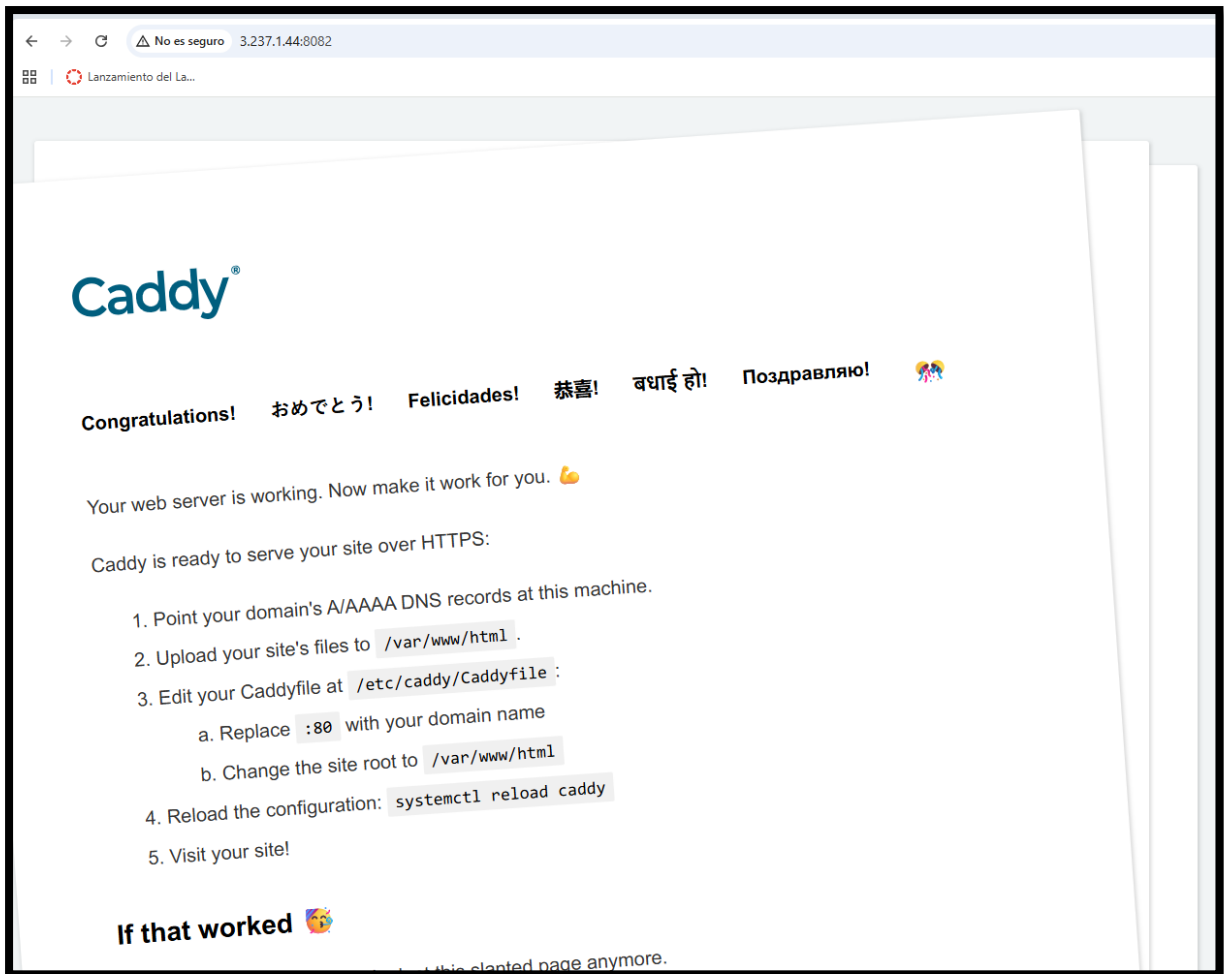
    Refer to the Caddy docs for more information:
    https://caddyserver.com/docs/caddyfile
}
```

The bottom of the terminal shows the nano editor's status bar with various keyboard shortcuts: `Help`, `Exit`, `Write Out`, `Read File`, `Where Is`, `Replace`, `Cut`, `Paste`, `Execute`, `Justify`, `Location`, `Go To Line`, `Undo`, `Redo`, `Set Mark`, and `Copy`. A status message `Read 26 lines` is also visible.

***Caddy funcionando puerto 8082***

### 5.3.2. Acceso desde el Navegador

- **URL de Acceso:** [http://TU\\_IP\\_PÚBLICA:8082](http://TU_IP_PÚBLICA:8082)
- **Explicación:** Se accede a Caddy por el puerto 8082. Se verificó que el servidor está respondiendo correctamente a las peticiones.



***Página de bienvenida de Caddy accesible a través del puerto 8082.***

## **5.4. Configuración SSL en Apache (Puerto 8443)**

### **5.4.1. Creación de Certificado Auto-firmado**

- **Acción:** Se utilizó `openssl` para generar un certificado y una clave privada auto-firmados, necesarios para probar el tráfico cifrado (HTTPS).
- **Comandos clave:** Se modificaron `/etc/apache2/ports.conf` (a 8443) y `/etc/apache2/sites-available/default-ssl.conf` para apuntar a los nuevos certificados y al puerto 8443.

GNU nano 7.2

```
# If you just change the port or add more ports  
# have to change the VirtualHost statement in  
# /etc/apache2/sites-enabled/000-default.conf
```

```
Listen 8443
```

```
<IfModule ssl_module>  
    Listen 443  
</IfModule>
```

```
<IfModule mod_gnutls.c>  
    Listen 443  
</IfModule>
```

```

GNU nano 7.2 /etc/apache2/sites-available/4
<VirtualHost *:8443>
    ServerAdmin webmaster@localhost

    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    #
    # SSL Engine Switch:
    # Enable/Disable SSL for this virtual host.
    SSLEngine on

    #
    # A self-signed (snakeoil) certificate can be created by installing
    # the ssl-cert package. See
    # /usr/share/doc/apache2/README.Debian.gz for more info.
    # If both key and certificate are stored in the same file, only the
    # SSLCertificateFile directive is needed.
    SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem
    SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key

    #
    # Server Certificate Chain:
    # Point SSLCertificateChainFile at a file containing the
    # concatenation of PEM encoded CA certificates which form the
    # certificate chain for the server certificate. Alternatively
    # the referenced file can be the same as SSLCertificateFile
    # when the CA certificates are directly appended to the server
    # certificate for convinience.
    #SSLCertificateChainFile /etc/apache2/ssl.crt/server-ca.crt

    #
    # Certificate Authority (CA):
    # Set the CA certificate verification path where to find CA
    # certificates for client authentication or alternatively one
    # huge file containing all of them (file must be PEM encoded)
    # Note: Inside SSLCACertificatePath you need hash symlinks
    # to point to the certificate files. Use the provided
    # Makefile to update the hash symlinks after changes.
    #SSLCACertificatePath /etc/ssl/certs/
    #SSLCACertificateFile /etc/apache2/ssl.crt/ca-bundle.crt

    #
    # Certificate Revocation Lists (CRL):
    # Set the CA revocation path where to find CA CRLs for client
    #

```

## Certificados SSL modificados

### 5.4.2. Acceso HTTPS desde el Navegador

- **URL de Acceso:** [https://TU\\_IP\\_PÚBLICA:8443](https://TU_IP_PÚBLICA:8443)
- **Explicación:** El acceso con <https://> y el puerto 8443 confirma que el certificado SSL se está utilizando, aunque el navegador muestre una advertencia porque el certificado no es emitido por una autoridad reconocida (es auto-firmado).



***Página por defecto de Apache accesible a través de HTTPS en el puerto 8443.***

## 6. Fase de Verificación Final de Servicios

### 6.1. Comprobación de Puertos de Escucha

- **Comando ejecutado:** `sudo netstat -tulpn` o comandos similares (`sudo ss -tulnp`) filtrando los puertos.
- **Explicación:** Se realizó una última verificación dentro de la Instancia EC2 para asegurar que todos los servicios (Apache, Nginx, Caddy, SSL) están escuchando simultáneamente en sus respectivos puertos (8080, 8081, 8082, 8443) y que no hay conflictos.

```
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 20 08:18:49 ip-172-31-76-39 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep caddy
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep apache2
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo netstat -tulpn | grep apache2
sudo: netstat: command not found
ubuntu@ip-172-31-76-39:~$ sudo ss -tulpn | grep apache2
tcp        LISTEN     0        128      *:8443      *:*      users:((("apache2",pid=979,fd=4),("apache2",pid=978,fd=4),("apache2",pid=559,fd=4))
tcp        LISTEN     0        128      *:443       *:*      users:((("apache2",pid=979,fd=6),("apache2",pid=978,fd=6),("apache2",pid=559,fd=6))
ubuntu@ip-172-31-76-39:~$ sudo ss -tulpn | grep nginx
tcp        LISTEN     0        128      0.0.0.0:8081 0.0.0.0:* users:((("nginx",pid=623,fd=5),("nginx",pid=622,fd=5),("nginx",pid=621,fd=5))
tcp        LISTEN     0        128      :::8081     ::::*    users:((("nginx",pid=623,fd=6),("nginx",pid=622,fd=6),("nginx",pid=621,fd=6))
ubuntu@ip-172-31-76-39:~$ sudo ss -tulpn | grep caddy
tcp        LISTEN     0        128      0.0.0.0:4096 0.0.0.0:* users:((("caddy",pid=522,fd=3))
tcp        LISTEN     0        128      :::4096     ::::*    users:((("caddy",pid=522,fd=7))
ubuntu@ip-172-31-76-39:~$
```

**Salida de netstat o ss confirmando que todos los puertos (8080, 8081, 8082, 8443) están en estado LISTEN.**

## 7. Conclusión y Reflexión sobre la Administración en la Nube

### 7.1. Lecciones Aprendidas de la Administración Remota

- **Comandos vs. Contexto:** Los comandos de Linux (como `apt install` o `nano`) no cambian, pero sí el contexto. Cada acción se realiza sobre un recurso remoto que implica responsabilidad real sobre la seguridad, disponibilidad y costes en AWS.
- **La Importancia del Security Group:** Aprendimos que la primera capa de *firewall* es la de AWS. Si el Security Group no está abierto, ningún servidor será accesible, sin importar lo que configuremos dentro de la EC2.
- **Seguridad de la Clave PEM:** La gestión segura de la clave PEM y sus permisos es el pilar de la autenticación remota en AWS, demostrando que la seguridad comienza en la estación de trabajo local del administrador.